Git Linux

https://www.digitalocean.com/community/tutorials/how-to-install-git-on-ubuntu-20-04-es

git
git --version
sudo apt install git
sudo apt autoremove

git config--global user.name “NombreUsuario”
git config--global user.email “EmailUsuario”
#git config --global init.defaultBranch main
#git config --list
nano ~/.gitconfig

#mkdir GitTest
#ls -al
#git init .
code .

Arranca Visual Code e instalar extensiones:
GitLens
GitGraph
GitExtensionPack
GitHub Pull Resquests and Issues
Git History
Git Conflict
GitHub Actios
GitHub Repositories
Docker
Docker Explorer
Docker Compose
Dev Containers
Dockerfile Linter
Kubernetes
Docker CLI
Docker Hub Tool
Sqlite
Sqlite Viewer

Reinicar VisualCode, deberíamos estar con el directorio y la rama principal.

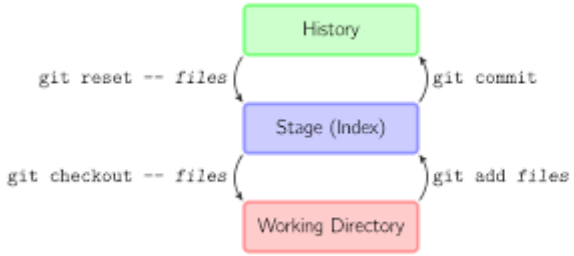
Git Windows

Web donde descargar el código actualizado: https://git-scm.com/
Chequeo que está bien instalada la aplicación
#git

Repositorio inicial
#git init .

Se puede borrar la carpeta y así ya se pierde el repositorio. Carpetas vacías no se guardan.
chequeo del estado, ficheros en el staging. Ficheros de texto los guarda bien, pero los binarios no los guarda bien, y las imágenes o pdfs los vuelve a subir.
#git status

Añadir fichero al staging (no es el repositorio, esto solo está en nuestro ordenador)
#git add "nombrefichero.txt"
#git status



Para añadir todos los ficheros del proyecto al staging
#git add .
#git status

Hacer el guardado, actualiza el repositorio común de todos los desarrolladores
#git commit

Hace falta registrar quien el desarrollador antes del guardado
#git --global user.name "nombre"
#git --global user.email "email"

Obligación: añadir algo para saber lo que se ha hecho
Recomendación: sólo una línea para resumen, solo una cosa, cambios atomizados, lo mínimo posible. Guarda tantas veces como sea necesario para seguir mejor el histórico
#git commit

Revisar log
#git log

Cambios añadidos
#git show

Volver a otra versión
#git checkout "hash"

Sacar un fichero del repositorio
#git reset "fichero"

Fichero .gitignore para quitar extensiones que no se guardará en el repositorio (*.pdf, *.json)
#git commit -m "comentario del commit para no tener que abrir el editor"

Ver diferencias
#git diff

Nos dice lo que ha habido en ese commit
#git show "hash"

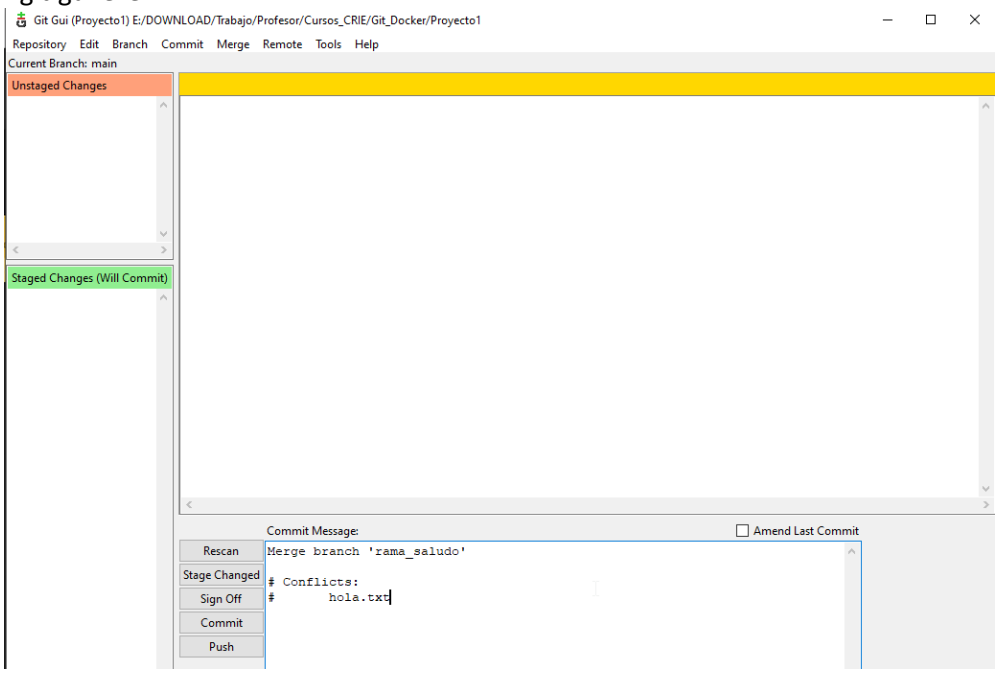
Para abrir nueva rama
#git switch -c "nombrerama"

Para abrir ramas si hemos vuelto atrás y hemos dejado cambios sin guardar, luego saltar a la rama
#git branch "nombrerama" "hash"
#git switch "nombrerama"
#git switch main

Mejor dejar de usar el git checkout para saltar ramas, es más simple usar switch, porque puede hacer más cosas
#git checkout main
Al revés no se puede, no se puede usar el git switch para ir a un punto anterior

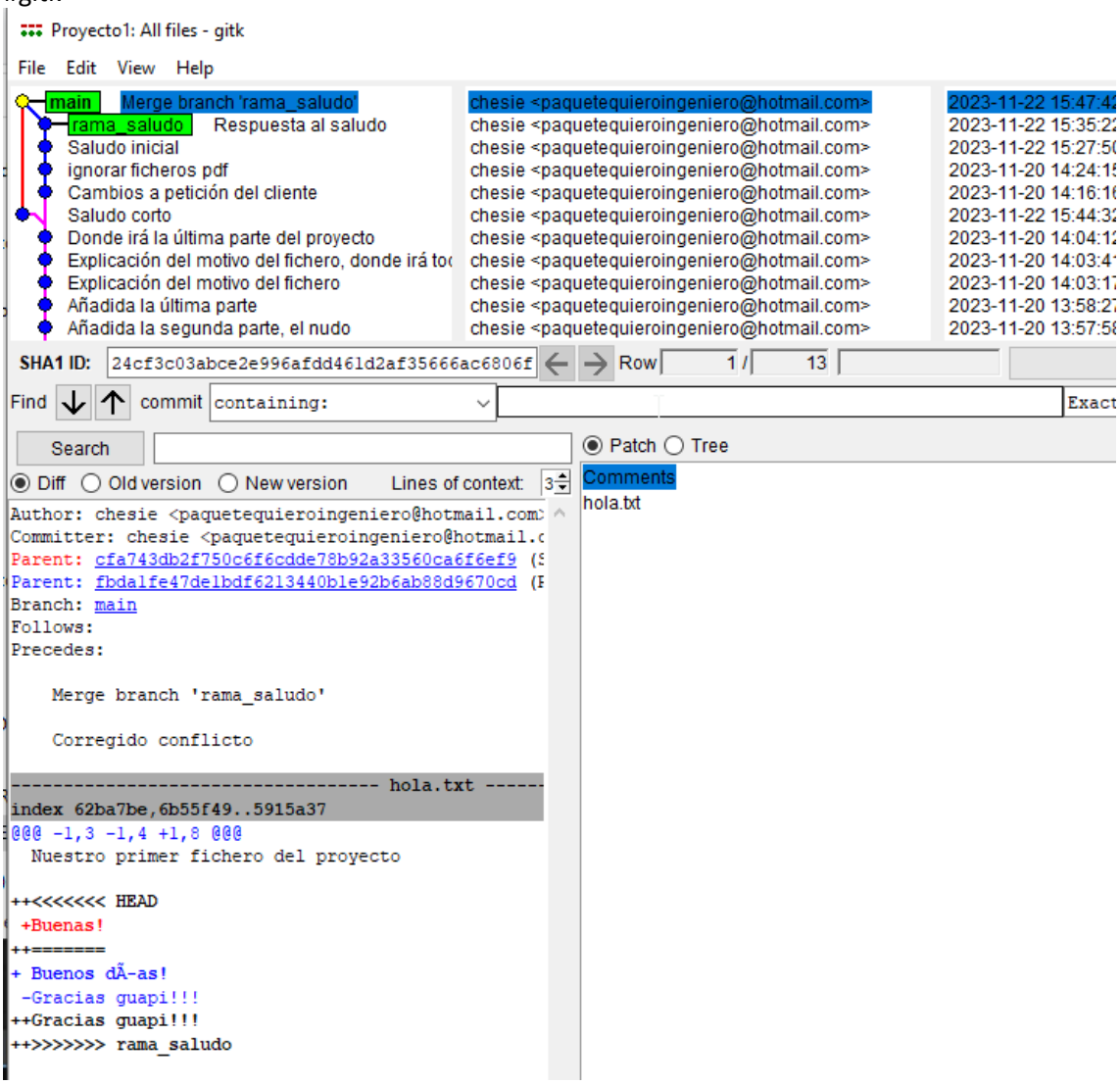
Para juntar rama al principal, o a otras ramas. Cuidado con los conflictos!
#git merge main
#git merge "nombrerama"
importante saber donde estamos, que es a donde traemos los cambios

Revisar conflictos
#git-gui.exe

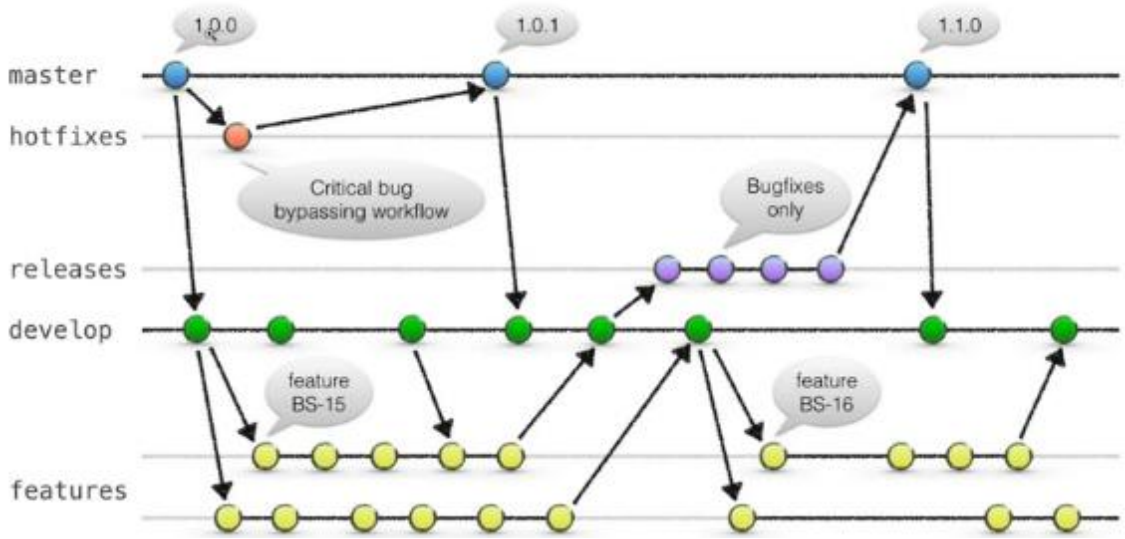


Entorno gráfico

#gitk



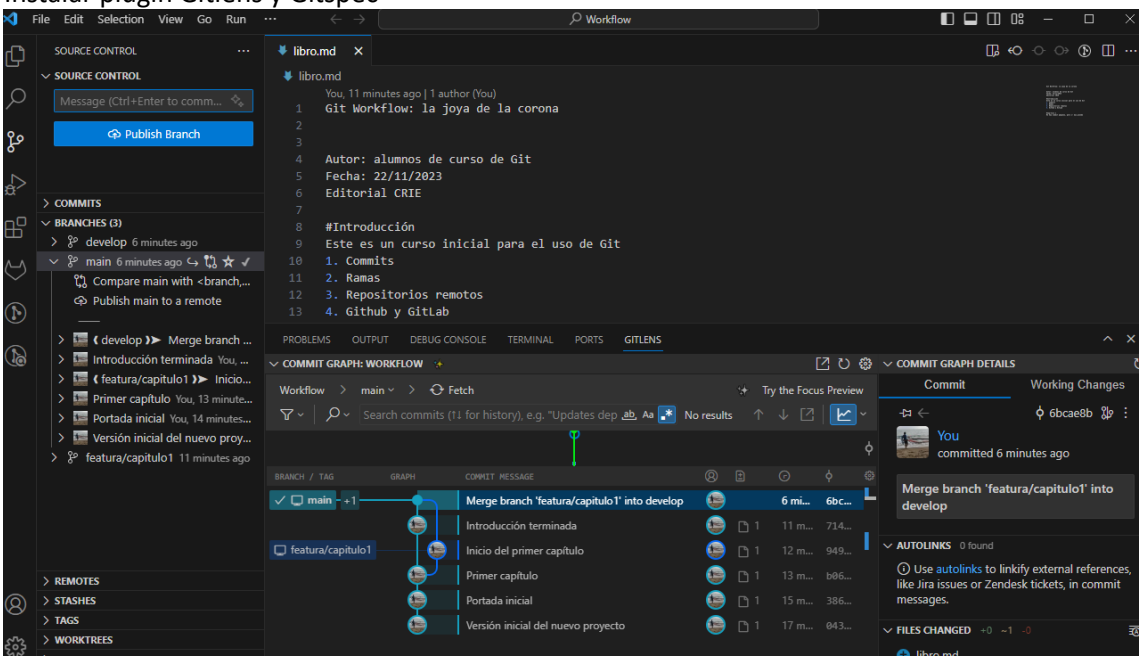
Workflow de git



El main siempre funciona, las pruebas con gaseosa … en la rama de desarrollo.
Hotfixes son excepcionales, corren mucha prisa. Para evitar efecto demo.
Las ramas de funcionalidades normalmente la tienen desarrolladores diferentes o bien, equipos de desarrolladores.

Las versiones se nombran en el main. Actualmente se usan fechas.
Se sigue usando la versión semántica 1.2.3 Major, minor y patch
Major -> cambios que dejan de ser compatibles con la BBDD o con datos anteriores, necesaria una migración.
Minor -> funcionalidades añadidas, sigue siendo compatible, no es necesaria una migración.
Patch -> cambios menores, normalmente estéticos o visuales, bugs pequeños.

Instalar plugin Gitlens y Gitspec



Después del merge, se suelen borrar las ramas para que esté limpito el proyecto.
branch –delete “nombrerama”

Para mostrar todas las ramas y su último commit
#git branch -v -a

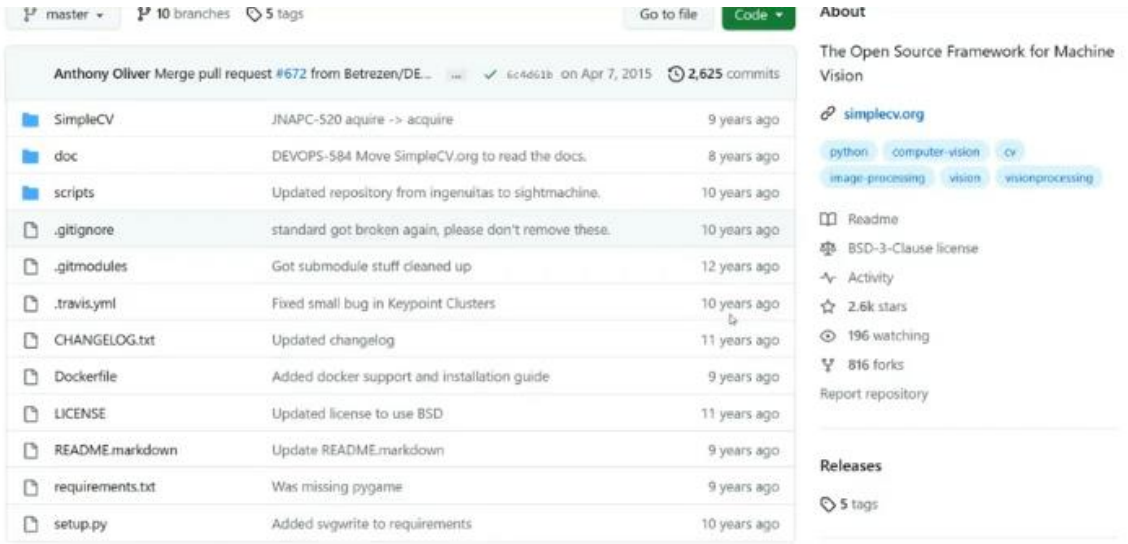
Recomendación de no borrar nada en la historia, se puede rehacer o traer cambios, pero no dejar ramas en el limbo, y mucho menos el main.

Desde Code se puede hacer Unstage para no validar unos datos en el add, es decir un add parcial, para llevar un orden al subir cositas poco a poco. Por comando, habría que añadir por número de líneas, siendo muy pesado.

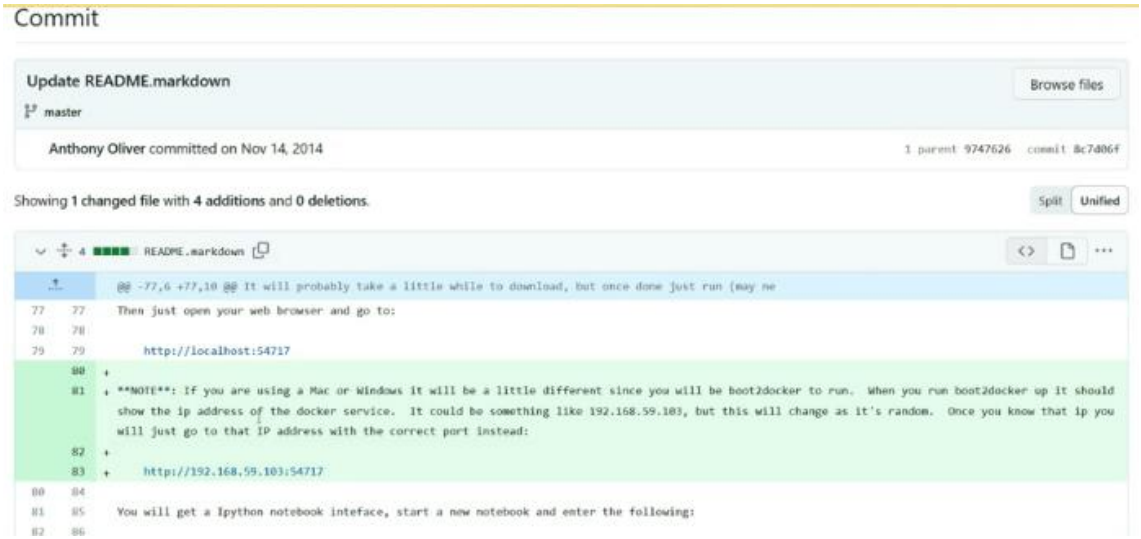
Lo normal es compartir con otros desarrolladores, se puede tener un repositorio único online con GitLab o GitHub. También se puede seguir con un repositorio distribuido con Git compartiendo y sincronizando los cambios.

https://github.com/ https://about.gitlab.com/

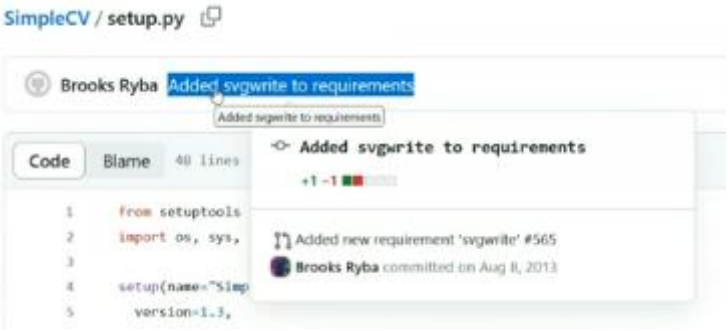
Repositorio de ejemplo en Github

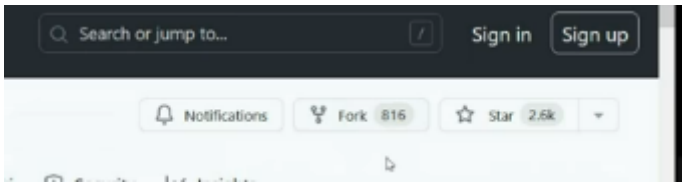


Ejemplo del último commit



Mensaje del commit y el autor del commit

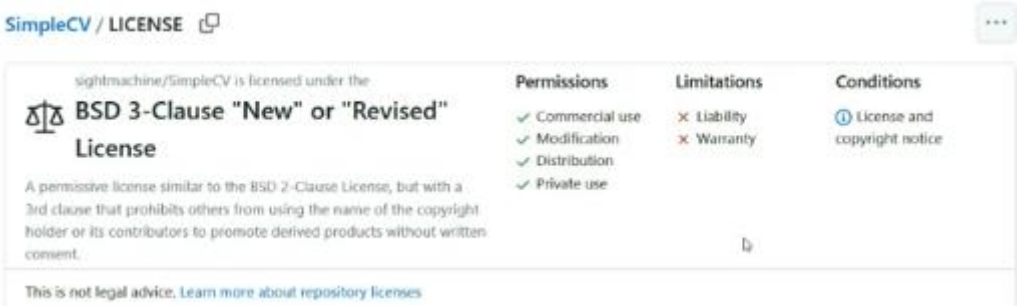




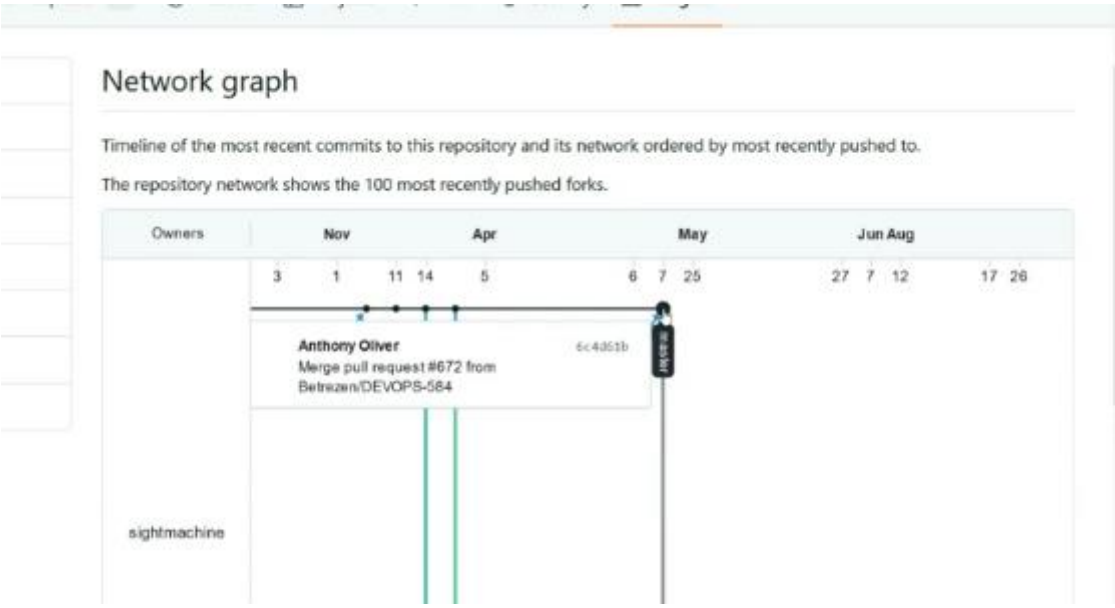
Fork -> otros desarrolladores han partido de este proyecto y están en otros proyectos con esa base. Puede ser que esté estancado y se quiera continuar por otra empresa.

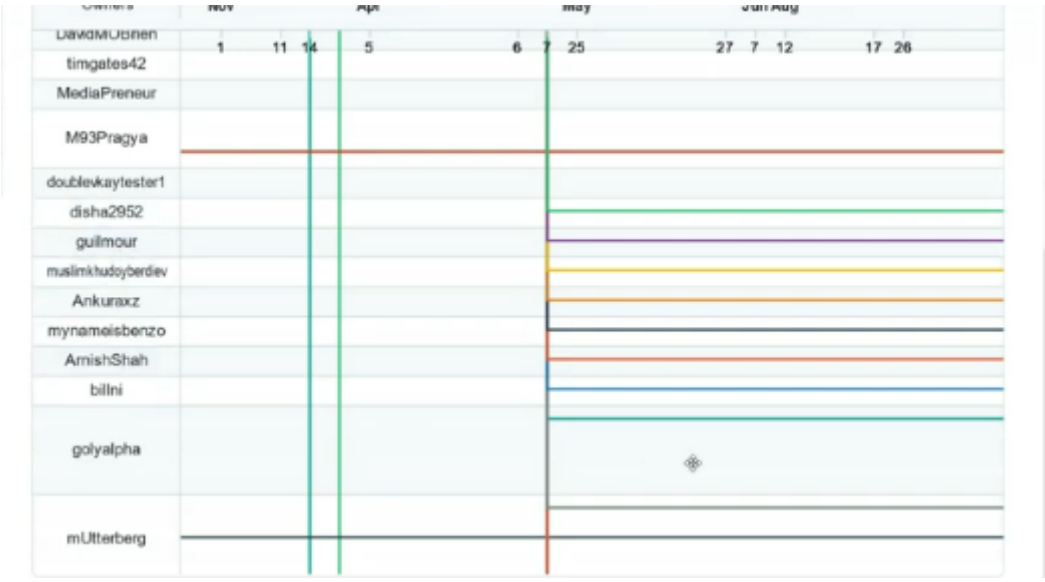
Star -> similar a “Me gusta” en rrss, indica la popularidad de un proyecto.

Es buena idea revisar la fecha del último commit que nos dará una idea del mantenimiento del proyecto. Así como la licencia para saber si podemos usar ese proyecto o esa biblioteca.



Un proyecto sin licencia, el derecho de autor es automático, es decir tiene copyright.





Se ven todos los fork que tiene el proyecto.

Se recomienda usar Github para proyectos abiertos, antiguamente dejaba hacer repositorios privados, pero ahora está cambiando su política y se están yendo los usuarios a GitLab.

Podemos descargar la propia plataforma Github a un servidor propio, y así no depender de tener conexión a Internet, y podemos trabajar con conexión en red en la oficina/empresa.

Podemos hacer un clone sin tener cuenta, pero para subir y colaborar, necesitamos cuenta, similar al git que necesitamos dejar constancia de los cambios con un usuario.

Desde git, nos ponemos en una ruta limpia

#git clone https://github.com/skills/introduction-to-github.git

Ahora tendremos los mismos ficheros y toda la historia. Para indicar el origen del repositorio

#git remote show

#git remote show origin

Opciones de fetch(download) y push(upload).

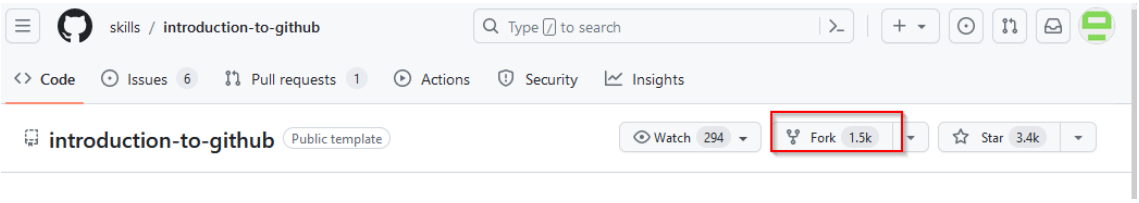
Merge a la rama main desde el main remoto

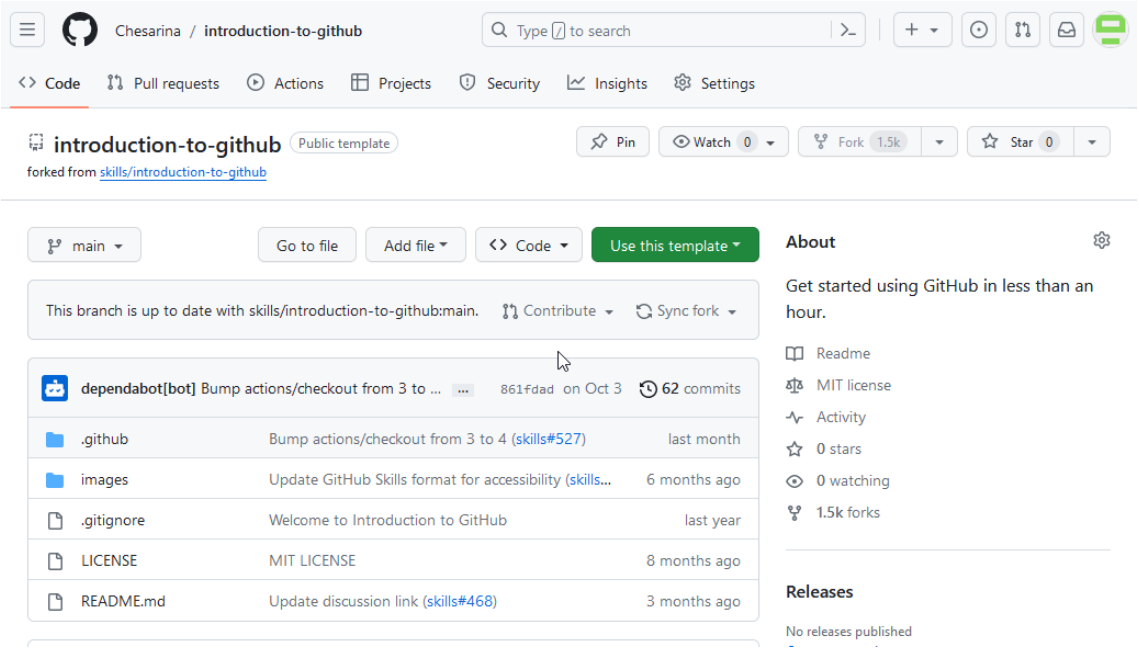
#git pull

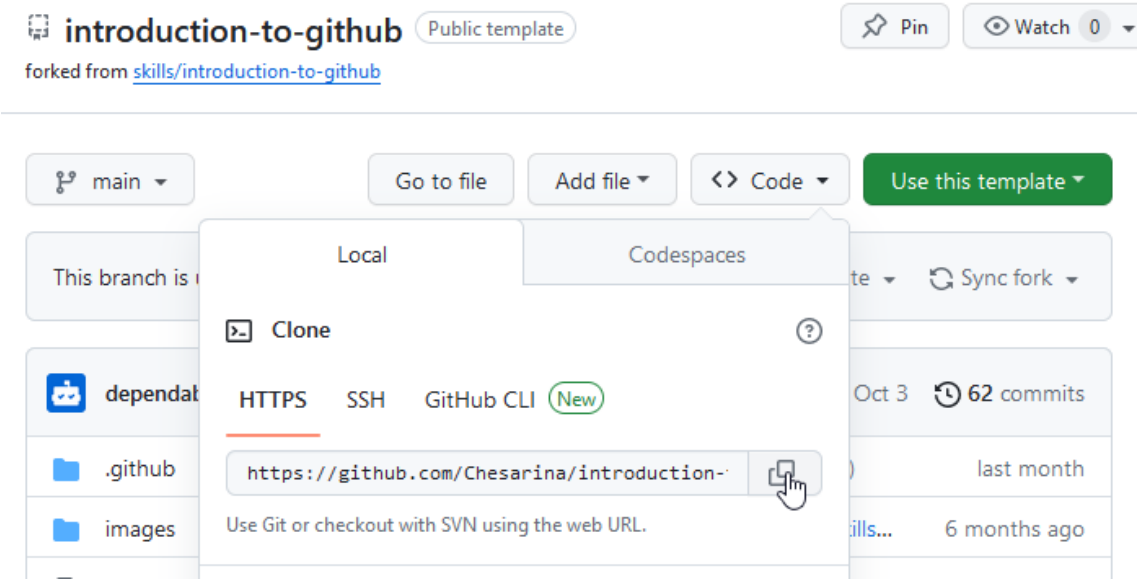
Merge al remoto desde el main en local, similar a publicar una rama en el VCode

#git push

Si intentamos subir, no nos dejará porque no tenemos permiso. Entonces creamos una cuenta e intentamos hacer un fork.







Eliminamos el anterior directorio y clonamos el nuevo para sustituirlo.

Hacemos un cambio en una rama nueva, develop.

Hacemos un add y un commit. Luego un merge a la main, y un publish al remoto.

Si no hemos hecho el merge, y estamos intentando hacer el push desde una rama, hará falta especificar desde que rama hacemos el push

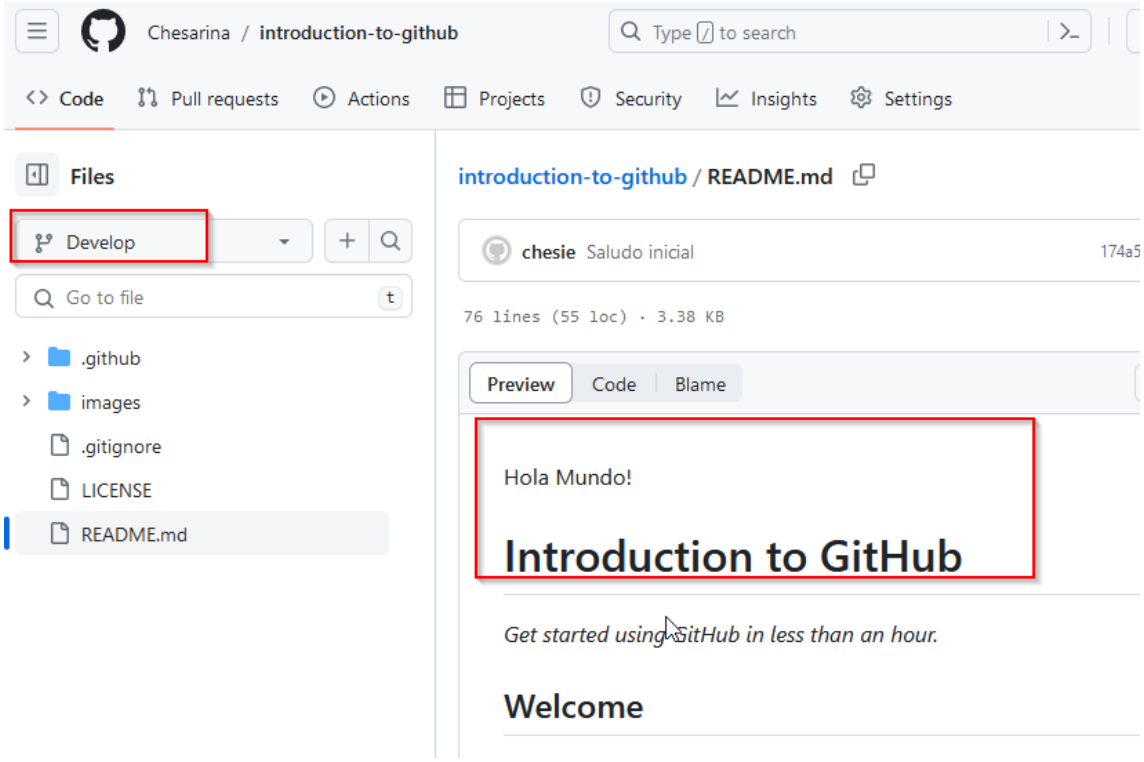
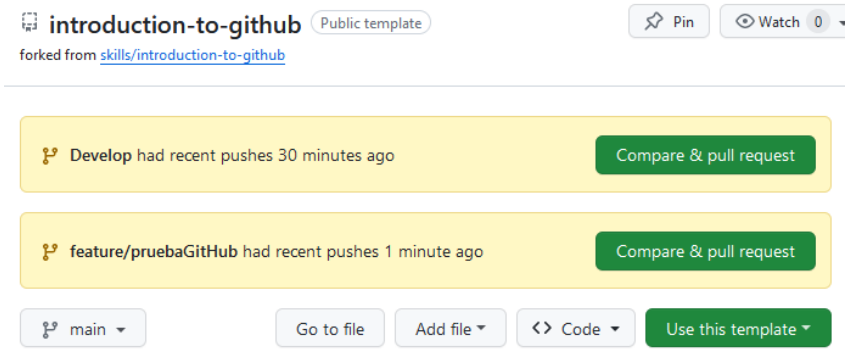
git push --set-upstream origin Develop

Puede ser que no se pueda conectar con un simple login y haga falta crear una clave ssh desde nuestra cuenta. Necesitamos tener instalado el ssh-keygen.

#ssh-keygen -t rsa -C “email”

Obtendremos una clave pública y otra privada, la pública es la que hay que subir a nuestra cuenta en Github.

Desde VisualCode podemos sincronizar nuestro desarrollo, hace un push&pull simultáneo.



Si creamos otra nueva rama, y la sincronizamos, veremos que nos ofrece un compare&pull, intentará pasarlo a la original, que como no es nuestro, no nos darán permisos. Forma de compartir nuestro desarrollo con otros repositorios de otros desarrolladores.